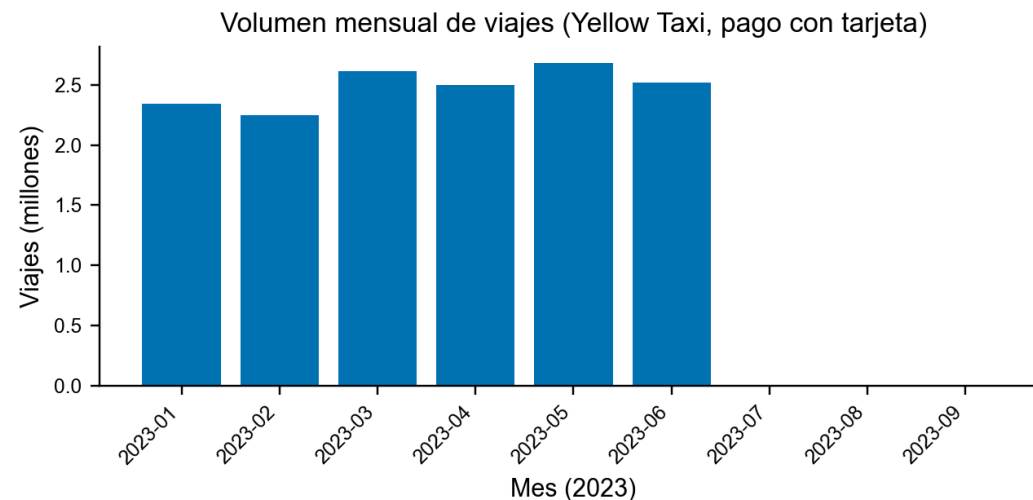


Predicción de propina baja en taxis de NYC con Apache Spark MLlib

Comparación de algoritmos de Machine Learning distribuido sobre Big Data

El problema y la necesidad

- La propina es una fracción clave del ingreso del conductor de taxi.
- ¿Podemos anticipar, al inicio del viaje, si dejará una propina baja?
- **Beneficiarios:**
 - Conductores: anticipar viajes de baja recompensa.
 - Plataformas/despacho: incentivos y asignación más justa.
- Reto técnico: decenas de millones de viajes → no cabe en una sola máquina con herramientas tradicionales.

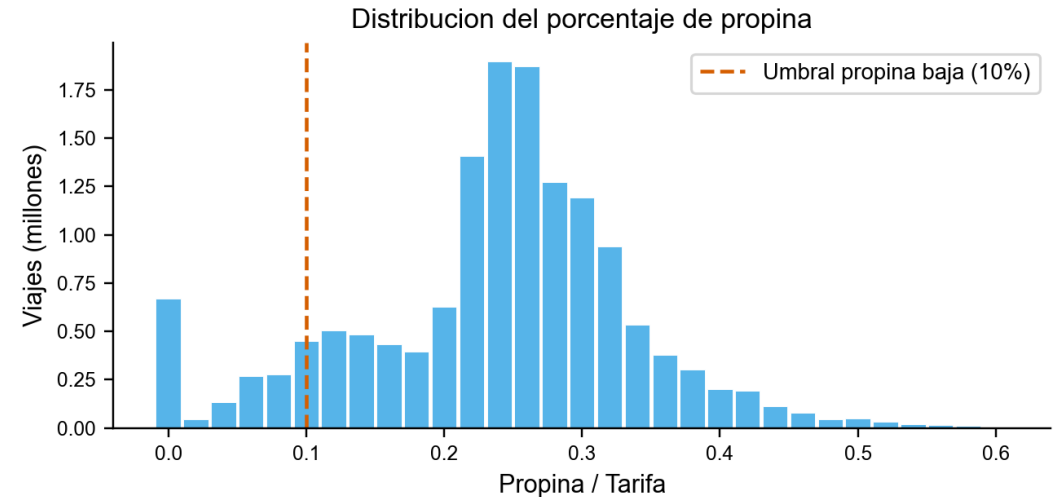


Propuesta de valor

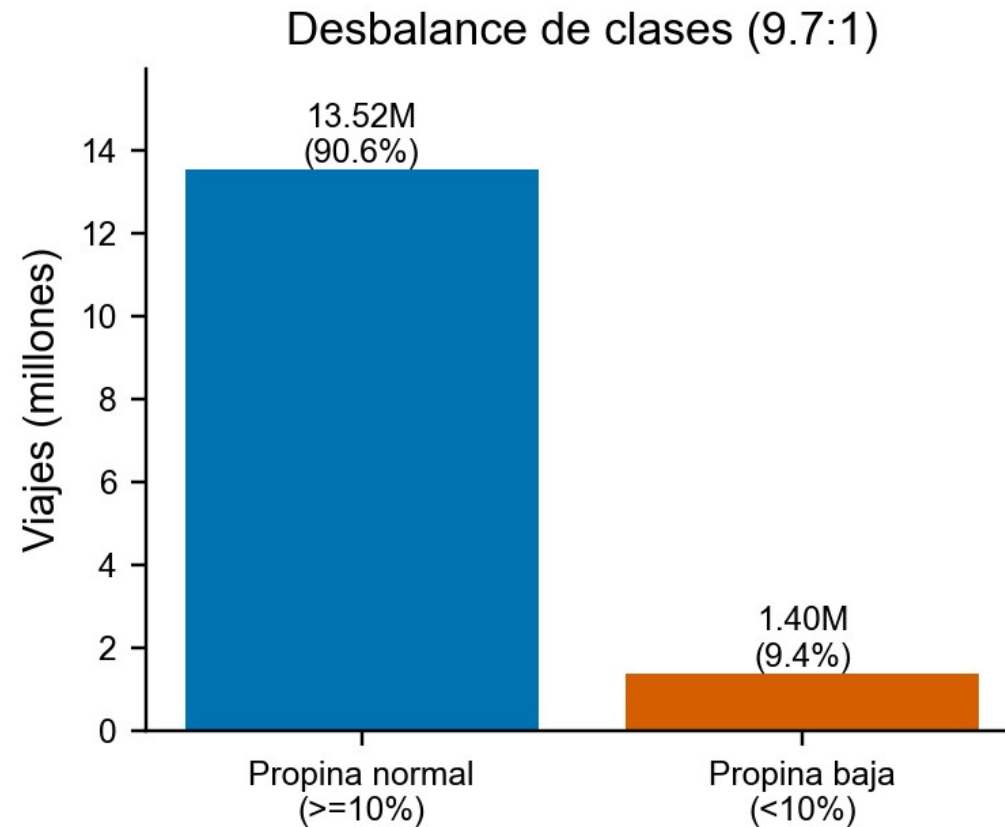
- Pipeline de Big Data reproducible, de extremo a extremo, sobre datos públicos reales.
- Compara 4 algoritmos distribuidos y mide su costo y escalabilidad.
- **Qué lo hace distinto:**
 - Métricas honestas bajo desbalance (no solo accuracy).
 - Evidencia cuantitativa de escalabilidad (datos y núcleos).
 - Decisiones documentadas (ADRs) y código abierto (repo público).

Los datos: NYC TLC Yellow Taxi

- 6 meses de 2023 · 19,493,620 viajes crudos → 14,912,381 tras limpieza.
- Formato Apache Parquet (columnar), ~2,5 M viajes/mes.
- **Las 5 V abordadas:**
 - Volumen: ~15 M de registros. Variedad: espacio, tiempo, economía.
 - Veracidad: solo pagos con tarjeta (la propina en efectivo no se registra).
 - Valor: anticipar la recompensa del conductor.



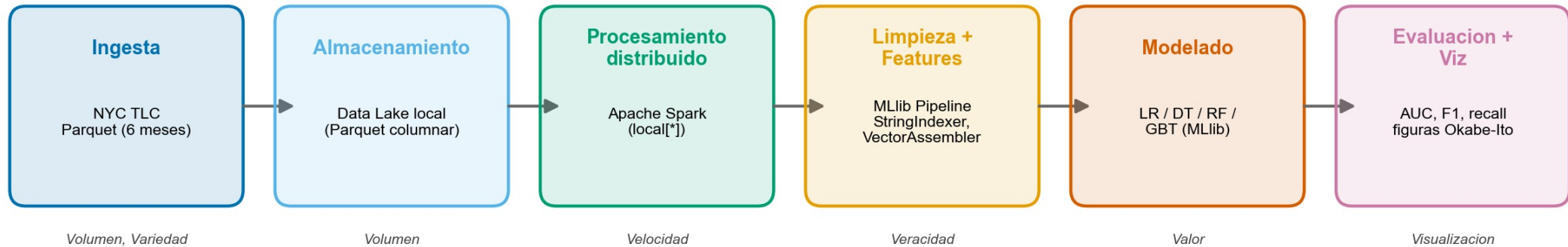
El reto central: desbalance de clases



La propina baja es minoritaria (9.4%), con un desbalance de 9.7:1. Un modelo trivial lograría ~90% de accuracy y 0% de recall útil.

Arquitectura: ciclo de vida del dato

Ciclo de vida del dato — Predicción de propina baja (NYC Taxi, Spark MLlib)



Etiquetas en cursiva: V de Big Data abordada en cada etapa

Ingesta → Almacenamiento → Procesamiento distribuido (Spark) → Features (MLlib) → Modelado → Evaluación y visualización.

Requerimientos de la solución

- **Funcionales:**

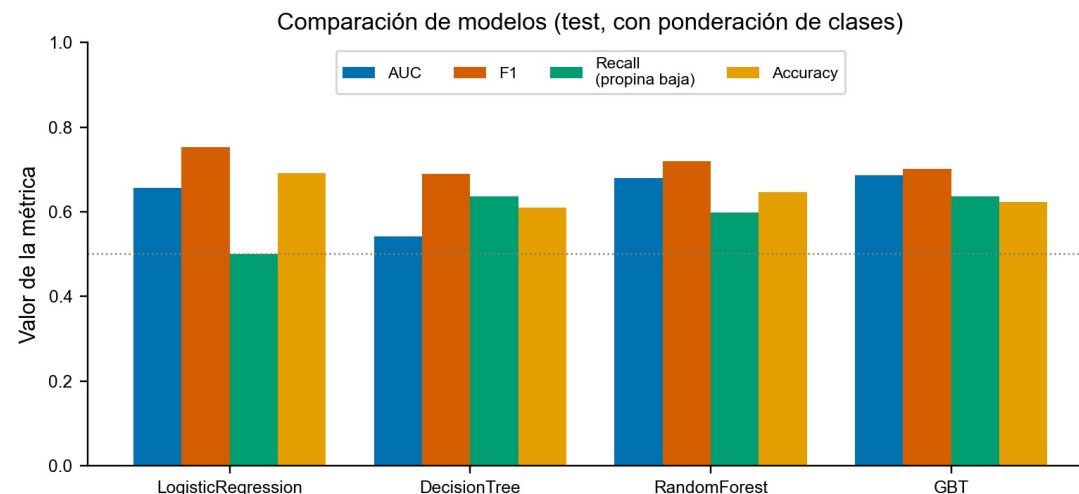
- Ingesta y normalización de Parquet con esquema variable.
- Limpieza, ingeniería de variables y etiquetado distribuido.
- Entrenamiento y comparación de modelos; reportes de métricas.

- **No funcionales:**

- Escalabilidad horizontal (datos y núcleos).
- Reproducibilidad (entorno y datos versionados).
- Veracidad y honestidad estadística de la evaluación.

Metodología (Spark MLlib)

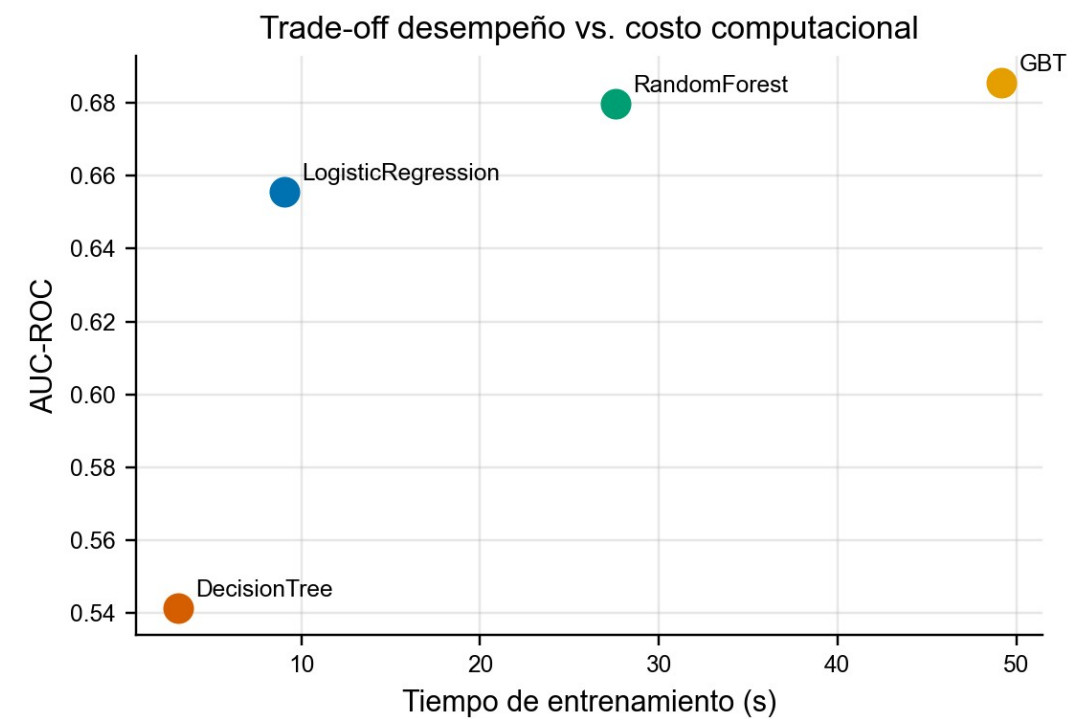
- Pipeline: StringIndexer → OneHotEncoder → VectorAssembler.
- 14 variables numéricas + 2 categóricas; se excluye la propina (anti-fuga).
- 4 modelos: Regresión Logística, Árbol, Random Forest, GBT.
- Desbalance: ponderación de instancias por clase (los 4 modelos).
- Split 80/20 idéntico para todos; AUC, F1, recall de la minoría.



Resultados: comparación de modelos

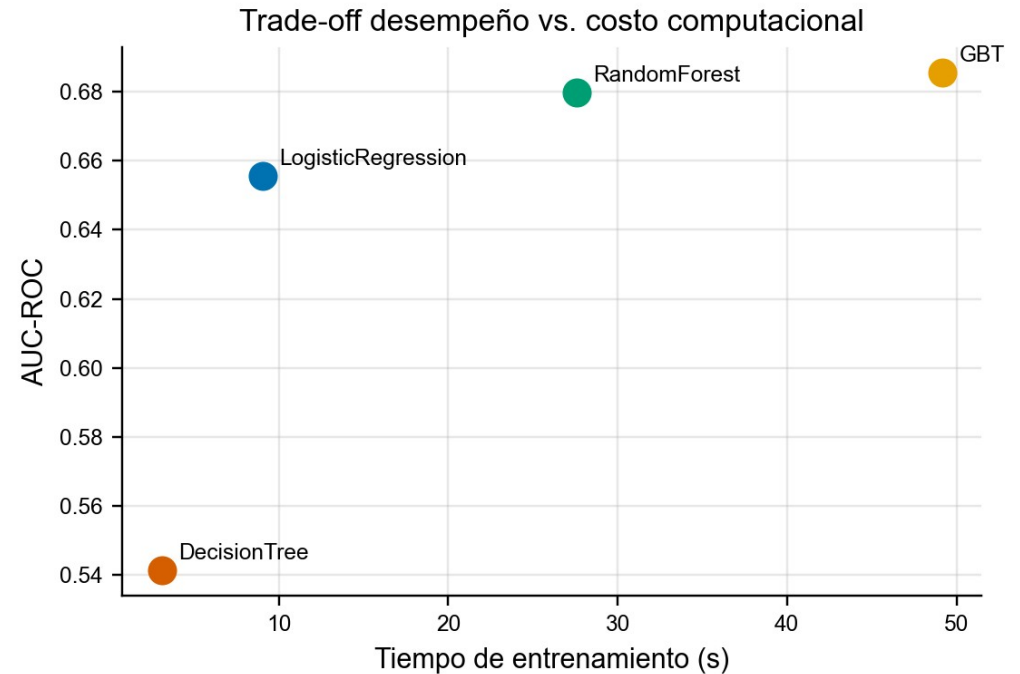
Modelo	AUC	F1	Acc.	Rec+	t(s)
LogisticRegression	0.656	0.753	0.691	0.501	9
DecisionTree	0.541	0.690	0.609	0.637	3
RandomForest	0.680	0.720	0.646	0.598	28
GBT	0.686	0.702	0.623	0.637	49

Con ponderación, los 4 modelos detectan la minoría. GBT y RF dan el mejor AUC; el árbol y GBT, el mejor recall — pero a costos muy distintos.

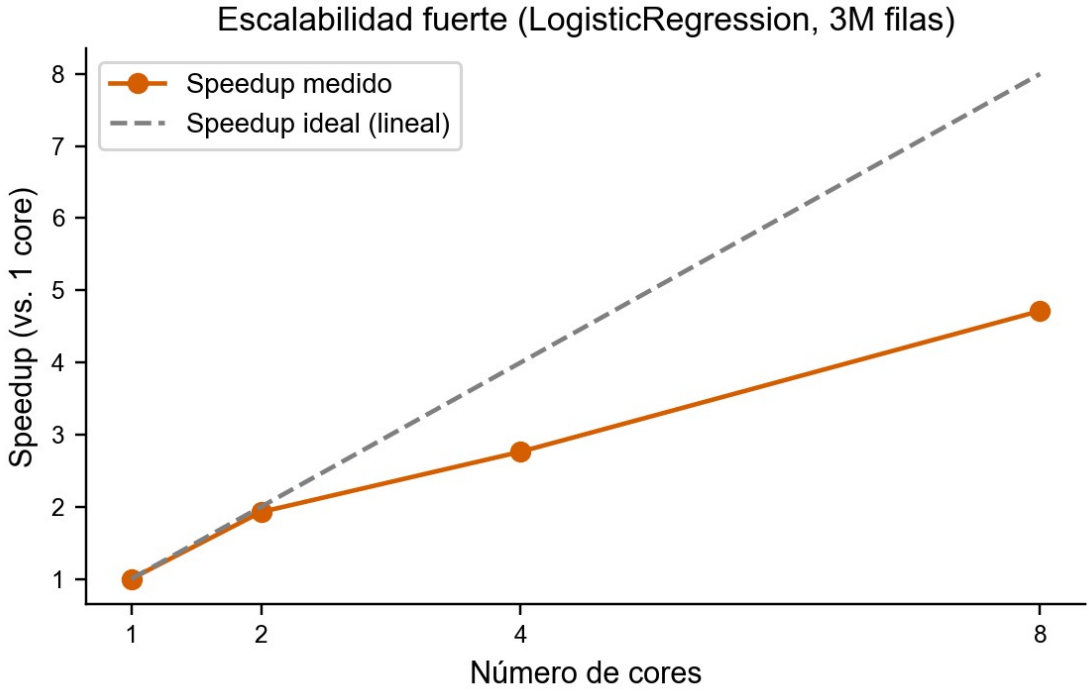
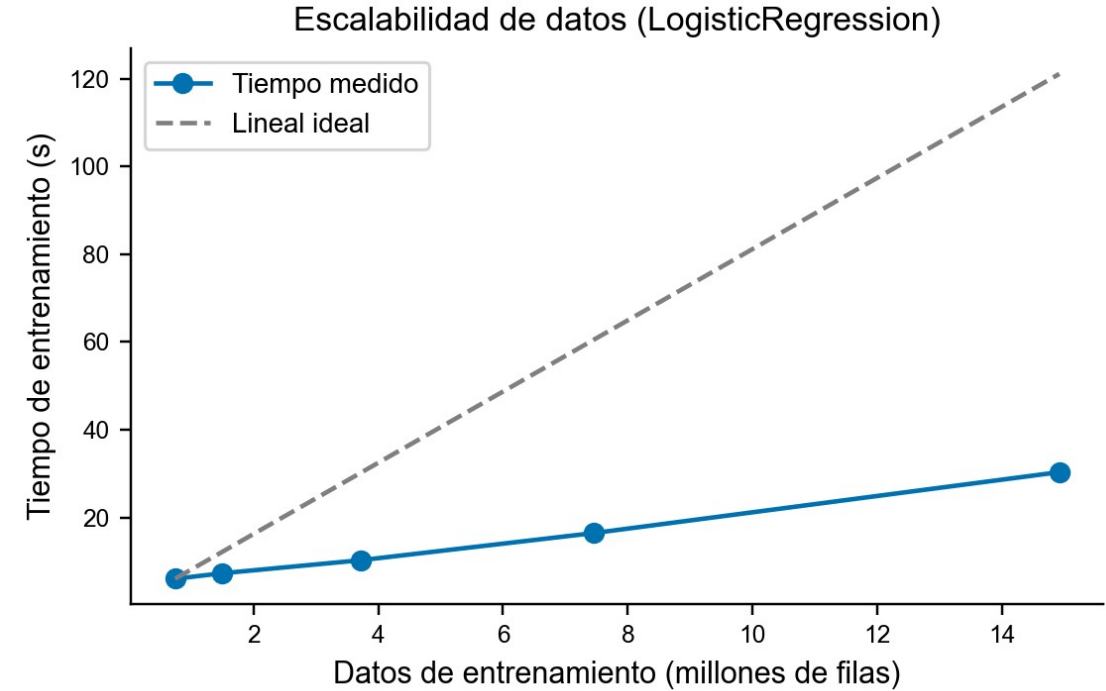


Hallazgo clave: la accuracy engaña

- Bajo fuerte desbalance, la accuracy NO es una métrica honesta.
- Un clasificador trivial lograría 90,6% de accuracy y 0% de recall útil.
- Por eso evaluamos con AUC y recall de la clase minoritaria.
- **No hay un único ganador:**
 - GBT: mejor desempeño (AUC 0.686, recall+ 0.64), mayor costo.
 - Random Forest: AUC casi igual (0.680) a la mitad del tiempo.
 - Regresión Logística: la más eficiente (9 s).



Escalabilidad del procesamiento distribuido



Escalabilidad de datos sublineal y speedup de 4.7× con 8 núcleos (ley de Amdahl).

Viabilidad e impacto

- **Escenarios de uso:**

- Incentivos dinámicos y asignación de viajes.
- Análisis del comportamiento de propina por zona/horario.

- **Limitaciones:**

- Sesgo: solo pagos con tarjeta; señal limitada de la propina.
- Ejecución local: no captura red/shuffle entre nodos.

- **Expansión:** clúster real (EMR/Databricks), datos contextuales (clima, tráfico), streaming.

Conclusiones

- Se comparó de forma justa el desempeño y el costo de 4 clasificadores distribuidos sobre ~15 M de viajes.
- Lección 1: en datos desbalanceados, elegir por AUC y recall, no por accuracy.
- Lección 2: Random Forest dio el mejor compromiso; la Regresión Logística, el menor costo.
- Lección 3: la escalabilidad confirma las ventajas y los límites (Amdahl) del cómputo distribuido.
- Código, datos y artículo: repositorio público reproducible.

¡Gracias!

Preguntas y discusión

Repositorio: github.com/nielspac177/big-data-nyc-taxi-mllib · nielspacheco1997@gmail.com